SMARTFARM

Rapport final de développement

Documentation du projet scolaire

Synthèse du développement, des vérifications et des limites du projet.

Document préparé à partir du fichier Markdown du projet.

Présentation et objectifs

SmartFarm est une application Django scolaire destinée à surveiller les bâtiments et espaces consacrés à l'élevage. Elle gère leur structure, les capteurs de température/humidité, les mesures historiques, les actionneurs et les commandes simulées. Elle ne gère pas les animaux individuellement.

Architecture et technologies

Le projet utilise Python 3.12.7, Django 5.2.19, Django REST Framework 3.16.1 et SQLite, avec templates HTML, CSS et JavaScript. Les applications sont `accounts`, `facilities`, `sensors`, `actuators`, `dashboard` et `simulator`. Les modèles principaux sont `Building`, `Space`, `Sensor`, `Measurement`, `Actuator` et `ActuatorCommand`. Les relations d'historique utilisent `PROTECT`; les entités liées sont désactivées/archivées lors d'une demande de suppression.

Fonctionnalités réalisées

- Connexion et déconnexion par le système Django, token API, profil et révocation du token.
- CRUD des bâtiments, espaces, capteurs et actionneurs, avec validation.
- Enregistrement, filtres, tri, pagination et consultation de l'historique des mesures.
- Commandes ON/OFF enregistrées avec historique et statut initial `PENDING`.
- Dashboard avec statistiques, dernières mesures, actionneurs et graphique sur 24 heures ou 7 jours.
- Simulateur Web qui génère des valeurs décimales et les transmet à l'API de mesures.
- Les endpoints API renvoient du JSON, y compris pour un chemin API inconnu.

API REST et sécurité

Les endpoints d'authentification sont `/api/v1/auth/token/`, `/api/v1/auth/me/` et `/api/v1/auth/token/revoke/`. Les autres groupes sont bâtiments/espaces, capteurs/mesures, actionneurs/commandes, dashboard et contrôle du simulateur. La liste détaillée figure dans `README.md` et `DOCUMENTATION_TECHNIQUE.md`.

Les endpoints métier nécessitent un token. Les pages Web utilisent les sessions Django; les formulaires et les actions AJAX du simulateur sont protégés par CSRF. Les capteurs inactifs, espaces inactifs, relations incorrectes et valeurs hors limites physiques sont refusés côté serveur. Les mesures et commandes ne sont pas supprimables par leur API; les entités qui les référencent sont conservées.

Les commandes d'actionneur sont des enregistrements simulés : elles ne pilotent aucun équipement physique. Aucun MQTT, ESP32, PostgreSQL, Docker, Redis, Celery ou système d'alertes n'est inclus.

Simulateur IoT

La page `/simulator/` démarre et arrête une tâche threadée en mémoire, une par utilisateur et par processus Django. Elle revérifie côté serveur le bâtiment, l'espace, l'activité et l'appartenance des capteurs. Le thread utilise le token DRF côté serveur pour appeler `POST /api/v1/measurements/`; le token n'est jamais exposé dans le navigateur. L'arrêt attend la fin d'un éventuel appel HTTP, limité à cinq secondes. Les boutons Démarrer et Arrêter ont été exercés dans le navigateur sur le serveur local : le compteur a atteint 4, puis est resté stable après l'arrêt et une attente supérieure à l'intervalle.

Le test navigateur a utilisé des données temporaires identifiées par `Phase9 Browser Test`; elles ont été supprimées ensuite et le serveur de test a été arrêté.

Tests et résultats réels

La suite complète a exécuté **137 tests, tous réussis** avec la commande demandée `\.venv\bin\python.exe manage.py test`. Elle couvre `accounts`, `facilities`, `sensors`, `actuators`, `dashboard` et `simulator`. Aucune suite n'a été supprimée ni aucun échec masqué.

- `manage.py check` : `System check identified no issues.`
- `manage.py makemigrations --check --dry-run` : `No changes detected.`
- `manage.py showmigrations` : toutes les migrations listées sont appliquées. `accounts`, `dashboard` et `simulator` n'ont pas de migration; `actuators` a `0001_initial`, `facilities` `0001_initial`, `sensors` `0001_initial` et `0002_measurement`; les migrations Django/auth/authtoken/contenttypes/sessions affichées sont marquées `[X]`.
- `findstatic js/simulator.js` trouve le fichier dans le dossier statique configuré.

Documentation créée

- `README.md` : présentation, architecture, environnement actuel, lancement, compte administrateur, authentification, routes API, simulateur, tests et limites.
- `DOCUMENTATION_TECHNIQUE.md` : applications, modèles, relations, vues, serializers, routes, authentification, erreurs et conservation des historiques.
- `GUIDE_UTILISATEUR.md` : parcours débutant pour les bâtiments, espaces, capteurs, mesures, actionneurs, dashboard et simulateur.
- `RAPPORT_FINAL.md` : synthèse des résultats du développement et vérifications.

Problèmes corrigés pendant l'intégration

- Un dossier de fichiers statiques non déclaré empêchait le navigateur de charger le JavaScript du simulateur; `STATICFILES_DIRS` a été configuré et la sélection bâtiment/espace a été vérifiée.
- Une URL API inconnue pouvait produire une page 404 HTML; un fallback 404 JSON et un test de non-régression ont été ajoutés.
- Le renderer DRF est limité au JSON conformément au contrat de l'API.

Limites connues

- Le simulateur utilise un registre de threads en mémoire propre à un processus; un redémarrage perd l'état et interrompt la tâche. Il convient au serveur local de développement, pas à un déploiement multi-processus.
- L'application utilise SQLite et aucun service matériel/IoT réel.
- Les commandes ne font que consigner l'intention ON/OFF.
- Il n'existe pas de système d'alertes ni de rôles utilisateurs personnalisés.

Commandes de lancement actuelles

Dans PowerShell, depuis le dossier SmartFarm :

```
.\.venv\bin\Activate.ps1
python manage.py check
python manage.py migrate
python manage.py runserver
```

La base existante `db.sqlite3` est conservée. Créez un compte administrateur si nécessaire avec `python manage.py createsuperuser`.

Vérifications manuelles restantes

1. Se connecter avec le compte administrateur habituel dans le navigateur de l’utilisateur.
2. Vérifier que les bâtiments, espaces et capteurs attendus dans la base SQLite apparaissent dans les listes.
3. Si Django est lancé sur un autre port que 8000, régler `SMARTFARM_API_BASE_URL` puis refaire un cycle Démarrer/Arrêter.

Ces contrôles portent sur les identifiants et données locaux de l’utilisateur; ils ne bloquent pas les résultats automatisés ci-dessus.